

Gradient Descent Variants

Gradient descent is the cornerstone optimization algorithm in deep learning, with different variants offering trade-offs between convergence speed, stability, and computational efficiency.

Stochastic Gradient Descent

SGD updates parameters using the gradient from **one datapoint (or a small batch)** at a time, rather than the entire dataset.

Walking downhill in the fog:

- Each step is guided by the slope at one random spot
- Path is noisy and zig-zagging, but on average moves downhill
- Can escape local minima due to stochasticity

Weight Update Rule:

$$\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} L_i(\theta)$$

Where:

- θ : Model parameters
- η : Learning rate
- $\nabla_{\theta} L_i(\theta)$: Gradient of loss for one sample/batch

Example

Loss Function: $L(w) = (w-3)^2$

Gradient: $dL/dw = 2(w-3)$

Iteration Steps (starting at $w=10$, $\eta=0.1$):

1. $w \leftarrow 10 - 0.1 \times 2(10-3) = 10 - 1.4 = 8.6$
2. Next datapoint $\rightarrow$ update again
3. Over time, w zig-zags toward optimal value of 3

Momentum Gradient Descent

Momentum adds a **velocity term** that accumulates past gradients, smoothing updates and accelerating convergence in consistent directions.

A ball rolling downhill:

- Plain SGD: ball moves only based on current slope $\rightarrow$ zig-zags
- Momentum: ball builds speed $\rightarrow$ smoother, faster descent along valleys
- Inertia helps overcome small bumps and flat regions

Velocity Update:

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} L_i(\theta)$$

Weight Update:

$$\theta \leftarrow \theta - v_t$$

Where:

- v_t : Velocity at time t
- γ : Momentum coefficient (typically 0.9)
- η : Learning rate

Example

Scenario: Gradients are consistently positive $[5, 4, 3]$ with $\eta=0.1$, $\gamma=0.9$

Iteration Steps:

1. $v_1 = 0.9 \times 0 + 0.1 \times 5 = 0.5$

- 2. $v_2 = 0.9 \times 0.5 + 0.1 \times 4 = 0.45 + 0.4 = 0.85$
- 3. $v_3 = 0.9 \times 0.85 + 0.1 \times 3 = 0.765 + 0.3 = 1.065$

Observation: Velocity grows → faster movement toward minimum

Comparative Analysis

Aspect	SGD	Momentum
Update Style	Based only on current gradient	Based on current + past gradients (smoothed)
Convergence Path	Noisy, zig-zag pattern	Smooth, accelerated trajectory
Convergence Speed	Slower, especially in valleys	Faster convergence in consistent directions
Local Minima Escape	Possible due to noise	Possible due to inertia
Hyperparameters	Learning rate (η)	Learning rate (η) + Momentum (γ)
Best Use Case	Small datasets, online learning, simplicity	Deep networks, ravines/narrow valleys

Python Implementation (TensorFlow/Keras)

◆ Basic SGD Implementation

```
import tensorflow as tf
import numpy as np
```

```

# Define a simple regression model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_shape=(5,)),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1) # Regression output
])

# Compile with SGD optimizer (no momentum)
model.compile(
    optimizer=tf.keras.optimizers.SGD(
        learning_rate=0.01,
        momentum=0.0 # Explicitly no momentum
    ),
    loss='mse',
    metrics=['mae']
)

# Generate dummy training data
X_train = np.random.randn(1000, 5).astype(np.float32)
y_train = np.random.randn(1000, 1).astype(np.float32)

# Train the model
print("Training with Vanilla SGD...")
history_sgd = model.fit(
    X_train, y_train,
    batch_size=32,
    epochs=10,
    validation_split=0.2,
    verbose=1
)

```

◆ SGD with Momentum Implementation

```

import tensorflow as tf
import numpy as np

# Define the same model architecture
model_momentum = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_shape=(5,)),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1)

```

```
] )

# Compile with SGD + Momentum
model_momentum.compile(
    optimizer=tf.keras.optimizers.SGD(
        learning_rate=0.01,
        momentum=0.9, # Enable momentum
        nesterov=False # Standard momentum
    ),
    loss='mse',
    metrics=['mae']
)

# Train with momentum
print("Training with SGD + Momentum...")
history_momentum = model_momentum.fit(
    X_train, y_train,
    batch_size=32,
    epochs=10,
    validation_split=0.2,
    verbose=1
)
```